HOA SEN UNIVERSITY
RESPECT DIVERSITY

Các thuật toán tìm kiếm (Search Algorithms)

Cấu trúc dữ liệu & Giải thuật – SW102DV01

(Data Structures & Algorithms)

Vũ Đình Khôi

Nội dung

- Đặt vấn đề tìm kiếm dữ liệu trong hệ thống thông tin
- Các giải thuật tìm kiếm
  - Tìm kiếm tuyến tính (linear/sequential search algorithm)
  - Tìm kiếm nhị phân (binary search algorithm)
  - Phân tích các thuật toán tìm kiếm (analysis)

Tham khảo

- Chapter 2. Jeffrey J. McConnell. (2001). *Analysis of Algorithms: An Active Learning Approach*. Jones and Bartlett Publishers
- Chapter 12. Robert Sedgewick. (1998). *Algorithms in C, Part 1-4, Third Edition*. Addison-Wesley

Giới thiệu

I find that I don't understand things unless I try to program them.

Donald E. Knuth

Professor Emeritus at Stanford University

If you want to **master something**, teach it

-- Richard Feynman

Giới thiệu

- Bài toán tìm kiếm (search)
  - Mục tiêu: tìm (find) các phần tử dữ liệu (items/records) có thông tin khóa (keys) khớp với (matching) một giá trị khóa cho trước (given key).
- Thường các dữ liệu (hoặc cơ sở dữ liệu)
  - Lưu trữ thông tin một lần nhưng có thể tìm kiếm nhiều lần
  - Thao tác cập nhật (update) ít hơn tìm kiếm
- Ứng dụng rộng rãi trong nhiều lĩnh vực/các hệ thông thông tin khác nhau
  - Giao dịch của khách hàng trong ngân hàng (bank)
  - Thông tin chuyến bay (airline)
  - Cỗ máy tìm kiếm (searching engine)
  - ...

Giới thiệu

- Bảng (table) được tổ chức trên B-Tree trong Microsoft SQL Server

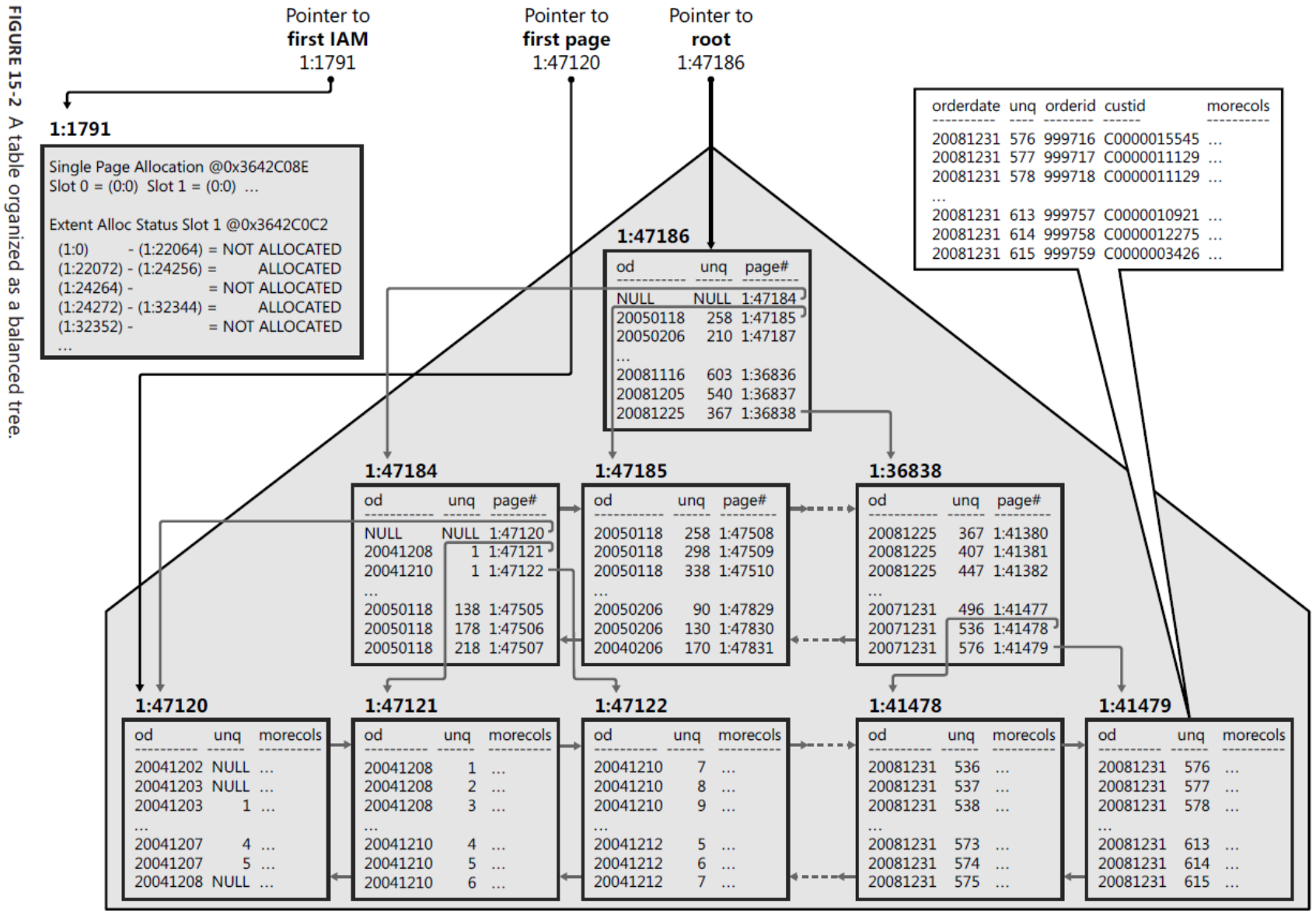


FIGURE 15-2 A table organized as a balanced tree.

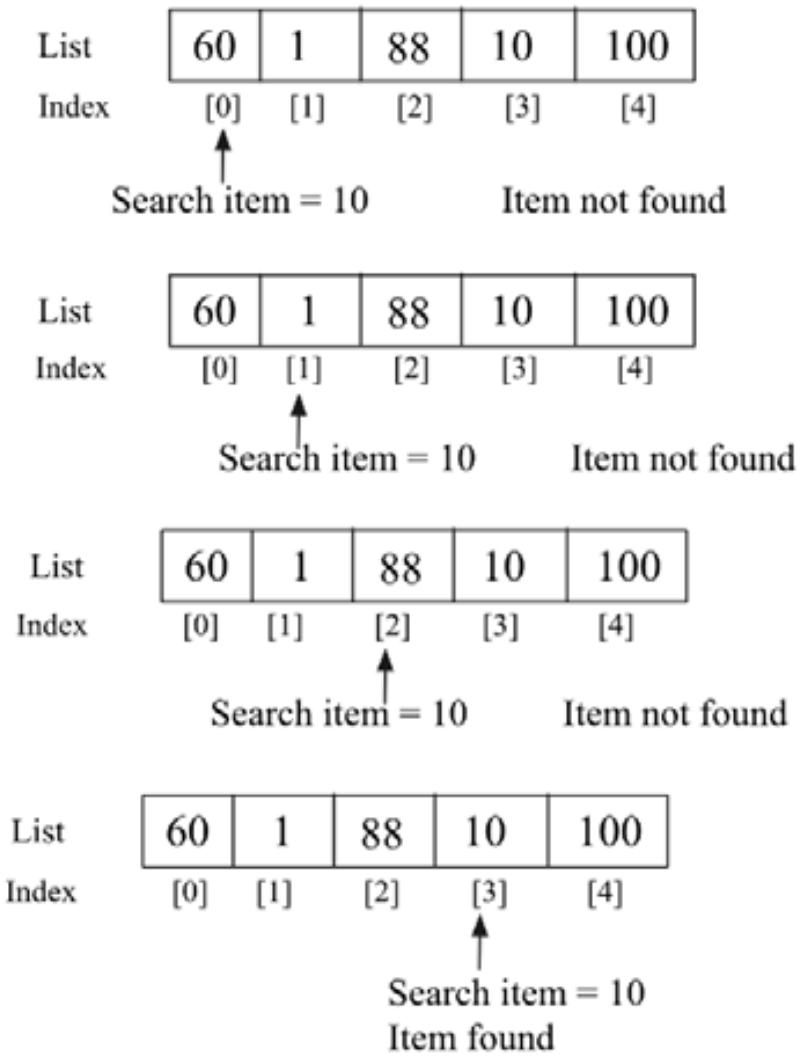
Giới thiệu

- Trong phạm vi bài này, chúng ta xem
  - Các phần tử dữ liệu (items/records) được lưu tại các vị trí liền kề nhau (adjacent locations) trong danh sách/mảng.
  - Gồm n phần tử (items)
  - Chỉ mục (index): 0 đến (n - 1)
  - Mỗi phần tử có thể chia thành nhiều trường dữ liệu (fields) nhưng chúng ta chỉ quan tâm đến trường chứa khóa (key).
  - Danh sách các phần tử có thể được chưa được sắp xếp/ngẫu nhiên (unsorted) hoặc sắp xếp (sorted) dựa trên giá trị khóa (key).
- Thuật toán tìm kiếm tuần tự (sequential search) phù hợp
  - Danh sách ngẫu nhiên
- Ngược lại, thuật toán tìm kiếm nhị phân (binary search) phù hợp
  - Danh sách được sắp xếp

Sequential/Linear search

- Tìm kiếm tuần tự (hay tuyến tính)
  - Ý nghĩa tuần tự hay tuyến tính là đi từ đầu đến cuối (hoặc từ cuối lên đầu) danh sách khi tìm kiếm.
- Bắt đầu từ phần tử đầu tiên của danh sách, so sánh lần lượt từng phần tử của danh sách với giá trị x cần tìm:
  - Nếu có phần tử bằng x, thuật toán dừng lại (thành công).
  - Nếu đến cuối danh sách mà không có phần tử nào bằng X, thuật toán dừng lại (không thành công).

Sequential/Linear search



Sequential/Linear search: analysis

- Trường hợp tốt nhất
  - phần tử đầu tiên có giá trị x
  - → Số lần so sánh: 1
- Trường hợp xấu nhất
  - phần tử cuối cùng có giá trị x hoặc không có phần tử có giá trị x
  - → Số lần so sánh: n
- Trường hợp trung bình
  - giả sử xác suất các phần tử có giá trị x là như nhau
  - → Số lần so sánh: (n + 1) / 2
- Độ phức tạp
  - T(n) = O(n)

Sequential/Linear search

```c
// Thuật toán tìm kiếm tuần tự (Sequential Search)
// (sequential_search.c)
#include <stdio.h>

// x: giá trị cần tìm (search key)
int sequential_search(int arr[], int n, int x) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == x) {
            return i;
        }
    }
    return -1;  // không tìm thấy
}

```

Trả về vị trí (index) của phần tử được tìm thấy

Sequential/Linear search

```c
void main() {
    int arr[] = {5, 2, 3, 7, 6, 1, 4, 0, 8, 9};
    int n = sizeof(arr) / sizeof(arr[0]);
    int x = 5;
    int result = sequential_search(arr, n, x);
    if (result == -1) {
        printf("Not found %d in list\n", x);
    } else {
        printf("Found %d at index %d\n", x, result);
    }
}
```

```
Found 5 at index 0
```

Sequential search: analysis

- Thao tác (operation) được thực hiện chủ yếu là các phép so sánh (comparisons)
- Trường hợp tốt nhất (best-case)
  - Phần tử/giá trị cần tìm (x) xuất hiện tại vị trí đầu danh sách → chỉ so sánh duy nhất 1 lần.
  - T(n) = O(1)
- Trường hợp xấu nhất (worst-case)
  - Phần tử cần tìm nằm cuối danh sách → cần thực hiện n bước so sánh để đi đến cuối.
- T(n) = O(n)

Sequential search: analysis

- Trường hợp trung bình (average-case)
  - Giả sử luôn tìm thấy x trong danh sách
  - Số thao tác trung bình để tìm thấy x tại vị trí ngẫu nhiên trong danh sách là bao nhiêu?
- Nhớ lại bài toán tính xác suất hoặc chi phí trung bình/thời gian trung bình

$$A(n) = \sum_{i=1}^{m} p_i \times t_i$$

  - n: kích thước (size) của input
  - m: số input classes
  - p_i: xác suất để một input rơi vào nhóm i
  - t_i: thời gian thuật toán thực thi cho một input thuộc nhóm i

Sequential search: analysis

- Xác suất tìm thấy x ở mỗi vị trí trong mảng a là như nhau: 1/n
- Xét mảng a gồm n phần tử, nếu thời gian xử lý tìm kiếm cho mỗi phần tử là t → phần tử thứ nhất là t, phần tử thứ 2 là 2*t, ..., phần tử i là i*t...
- Xem t là 1 đơn vị thời gian xử lý, Do đó

$$A(n) = \sum_{i=1}^{m} p_i \times t_i = \sum_{i=1}^{n} \frac{1}{n}(i \times t) = \frac{1}{n}\sum_{i=1}^{n} i = \frac{1}{n} \times \frac{n(n+1)}{2} = \frac{n+1}{2}$$

- Nếu xét đến cuối dãy nhưng không tìm thấy x thì sao?
  - Tốn thêm 1 thao tác và (n * t) thời gian để xử lý

$$A(n) = \frac{1}{(n+1)}\left(\sum_{i=1}^{n} i + n\right) \approx \frac{n+2}{2}$$

- Vậy cả 2 trường hợp → $A(n) \approx n/2$

Binary search

- Điều kiện
  - Thực hiện trên mảng/danh sách đã được sắp xếp (sorted) theo thứ tự tăng dần (hoặc giảm dần). → hay danh sách có thứ tự
- So sánh giá trị muốn tìm X với phần tử nằm ở vị trí giữa của danh sách:
  - Nếu bằng, tìm kiếm dừng lại (thành công).
  - Nếu X lớn hơn thì tiếp tục tìm kiếm ở phần danh sách bên phải phần tử giữa.
  - Nếu X nhỏ hơn thì tiếp tục tìm kiếm ở phần danh sách bên trái phần tử giữa.

Binary search

- Thực hiện trên dãy/mảng/danh sách đã được sắp xếp (sorted) theo thứ tự tăng dần (hoặc giảm dần).
  - So sánh x với phần tử giữa dãy a[middle].
  - Nếu x = a[middle] → Found
  - Nếu x < a[middle] → x sẽ nằm trước/trái a[middle]
  - Ngược lại →x sẽ nằm sau/phải a[middle]
- Chia đôi chiều dài của dãy cho đến khi tìm thấy hoặc hết dãy (không tìm thấy
  - Nếu mảng có 8 phần tử → các lần so sánh để tìm thấy x tối đa: $\log_2 8 = \log_2 2^3 = 3$
  - Với n phần tử → $\log_2 n$

Binary search

If we want to search 43 in the given list

1	4	11	25	32	37	40	43	47	49	53	55

since 43>37
We look only at the second half

1	4	11	25	32	37	40	43	47	49	53	55

since 43>37
We now look only at the first half of the list

1	4	11	25	32	37	40	43	47	49	53	55

Search item 43 is found.
The function will return the index position

1	4	11	25	32	37	40	43	47	49	53	55

Binary search

```c
// Thuật toán tìm kiếm nhị phân (Binary Search)
// (binary_search.c)
#include <stdio.h>
// x: giá trị cần tìm (search key)
int binary_search(int arr[], int n, int x) {
    int left = 0;
    int right = n - 1;
    while (left <= right) {
        int mid = (left + right) / 2;
        if (arr[mid] == x) return mid;
        if (arr[mid] < x)
            left = mid + 1;
        else
            right = mid - 1;
    }
    return -1;  // không tìm thấy
}
```

Binary search

```
// Đệ quy
int recursive_binary_search(int arr[], int left, int right, int x) {
    if (left <= right) {
        int mid = (left + right) / 2;
        if (arr[mid] == x) return mid;
        if (arr[mid] < x)
            return recursive_binary_search(arr, mid + 1, right, x);
        else
            return recursive_binary_search(arr, left, mid - 1, x);
    }
    return -1;  // không tìm thấy
}
```

Binary search

```c
void main() {
    int arr[] = {2, 5, 6, 8, 10, 15, 16, 17, 18, 20};
    int n = sizeof(arr) / sizeof(arr[0]);
    int x = 6;
    int result = binary_search(arr, n, x);
    if (result == -1)
        printf("Not found %d in list\n", x);
    else
        printf("Found %d at index %d\n", x, result);

    result = recursive_binary_search(arr, 0, n - 1, x);
    if (result == -1)
        printf("Not found %d in list\n", x);
    else
        printf("Found %d at index %d\n", x, result);
}
```

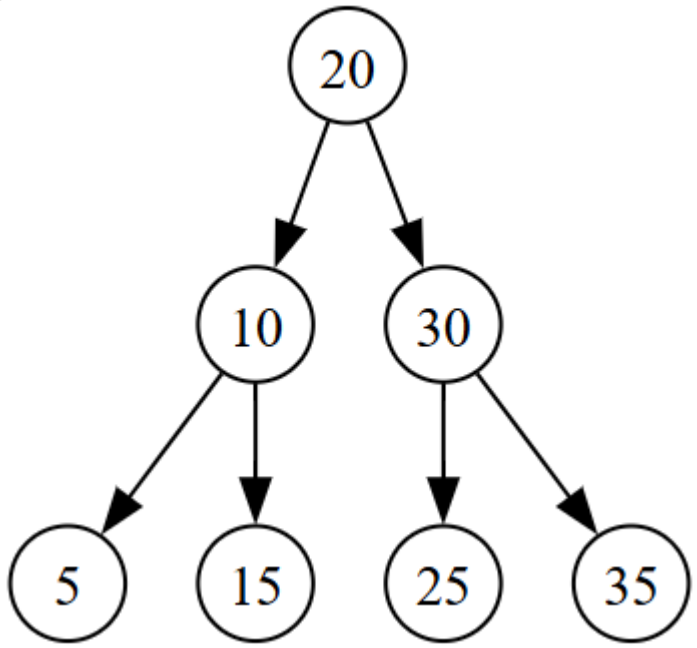
```
Found 6 at index 2
Found 6 at index 2
```

Binary search: analysis

- Thao tác (operation) được thực hiện chủ yếu là các phép so sánh (comparisons).

index	0	1	2	3	4	5	6
a[i]	5	10	15	20	25	30	35

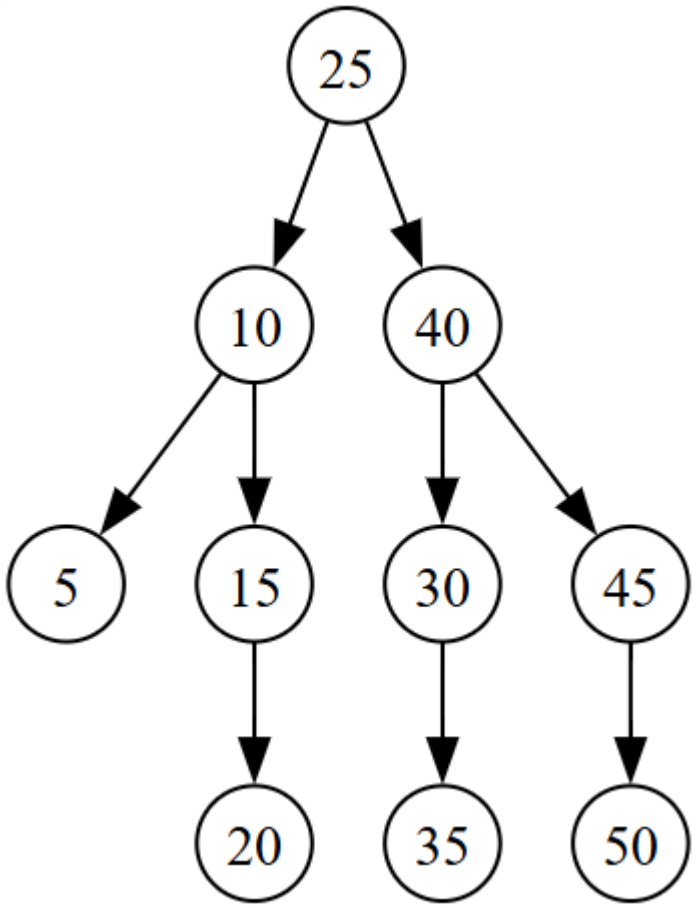
```
search(arr[], n = 7, x = 36)
step 1: m = (0 + 6) / 2 = 3
step 2: m = (4 + 6) / 2 = 5
step 3: m = (6 + 6) / 2 = 6

stop (vì left = 7, right = 6)
Tổng số bước: k = lg(n) + 1
= lg(7) + 1 = 2.8 + 1 = 3
```



Binary search: analysis

index	0	1	2	3	4	5	6	7	8	9
a[i]	5	10	15	20	25	30	35	40	45	50

```
search(arr[], n = 10, x = 51)
step 1: m = (0 + 9) / 2 = 4
step 2: m = (5 + 9) / 2 = 7
step 3: m = (8 + 9) / 2 = 8
step 4: m = (9 + 9) / 2 = 9
stop (vì left = 10, right = 9)
Tổng số bước: k = lg(n) + 1
= lg(10) + 1 = 3.3 + 1 = 4
```



Binary search: analysis

- Trường hợp tốt nhất (best-case)
  - Phần tử/giá trị cần tìm (x) xuất hiện tại vị trí giữa danh sách → chỉ so sánh duy nhất 1 lần.
  - T(n) = O(1)
- Trường hợp xấu nhất (worst-case)
  - Phần tử cần tìm nằm cuối danh sách → cần thực hiện $\log_2(n) + 1$ bước so sánh để đi đến cuối.
  - T(n) = O(log n)
- Trường hợp trung bình (average-case)
  - Hơi rắc rối chút ☺ → xem slide kế tiếp

Binary search: average-case analysis

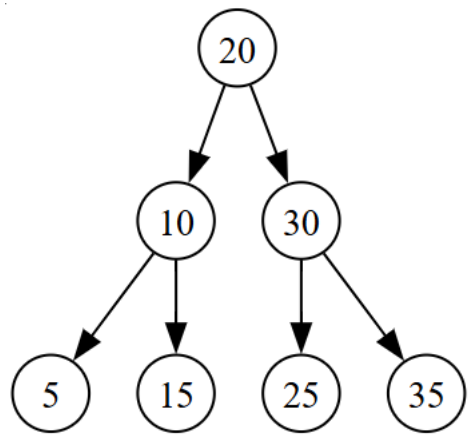
- Xem lại các hình bên, thời gian trung bình

$$A(n) = \sum_{i=1}^{n} p_i \times t_i$$

- Chuyển sang tính A(n) theo xác suất của các node trên cây:
  - Xác suất mỗi phần tử xuất hiện tại 1 vị trí bất kỳ trên mảng là $p_i = 1/n$
  - Mỗi cấp độ (level), mỗi bước i có tối đa $2^{(i-1)}$ node.
  - Thời gian xử lý cho mỗi cấp/bước là như nhau, bước 1 tốn t thời gian, thì bước 2 tốn 2*t, bước 3 là 3*t (vì muốn tới bước 3 phải qua bước 1 và bước 2)
    → $t_i = p_i \times t$

index	0	1	2	3	4	5	6
a[i]	5	10	15	20	25	30	35

```
search(arr[], n = 7, x = 36)
step 1: m = (0 + 6) / 2 = 3
step 2: m = (4 + 6) / 2 = 5
step 3: m = (6 + 6) / 2 = 6

stop (vì left = 7, right = 6)
Tổng số bước: k = lg(n) + 1
= lg(7) + 1 = 2.8 + 1 = 3
```



Binary search: average-case analysis

- Thời gian trung bình tính theo cây

$$A(n) = \sum_{i=1}^{k} \frac{1}{n} \times 2^{i-1} \times (i \times t)$$

  - Với k là tổng số bước, cho t = 1 (đơn vị thời gian)

$$A(n) = \frac{1}{n} \sum_{i=1}^{k} i \times 2^{i-1} = \frac{1}{n} \times \frac{1}{2} \sum_{i=1}^{k} i2^{i}$$

  - Biến đổi một chút (tham khảo textbook) ta được

$$A(n) \approx \log_2(n+1) - \frac{1}{2}$$

- Do đó
  - A(n) = O(log n)

Binary search: average-case analysis

- Ví dụ mảng gồm n = 7 phần tử được sắp xếp
- Tìm kiếm tuần tự:

$$A(n) \approx \frac{n}{2} = 3.5$$

- Tìm kiếm nhị phân (với k = lgn + 1 = 3):

$$A(n) \approx \frac{1}{2n}\sum_{i=1}^{k} i2^i = \frac{1}{2 \times 7}(1 \times 2^1 + 2 \times 2^2 + 3 \times 2^3) = \frac{34}{14} = 2.43$$

- Tính theo công thức

$$A(n) \approx \log_2(n+1) - \frac{1}{2} = 3 - 0.5 = 2.5$$

- Rõ ràng, tìm kiếm nhị phân hiệu quả hơn nhiều nếu n tăng nhanh.